

Mobility Lens Assignment 1 Report

Details:

Name: Jerin Joseph

BU ID: U11191999

Application Package Name: com.jerinjoseph.mobilitylens

Emulator Model: Pixel 3a Android 8.1 ("Oreo") | x86

API Level Used: API 27

Min SDK: 26 (API 26 ("Oreo"; Android 8.0))

This is the oldest version supported by the application. The package installer refuses to add any packages below this sdk. This is relevant during the install time by determining which all devices can install this application.

I chose this version because it has a huge coverage support (95%+). I didn't go below that cause I felt there is not much increase in the coverage support and going below would also mean that older library version could also be added in the project which could have some security issues.

Target SDK: 37 (API 37 ("CinnamonBun"; Android 17.0))

This sdk version determines the type of behaviour required for the application. It assumes that the application is intended to run on sdk 37 and provides only the relevant permissions/services for this sdk version

Compiled SDK: 37 (API 37 ("CinnamonBun"; Android 17.0))

The version the application is compiled against. It is important during builds and is not written in any of the build files. It determines which classes and methods can be referenced depending on the sdk version selected.

Main Activity

Location: app/src/main/java/com/jerinjoseph/mobilitylens/MainActivity.kt

It is the activity component that opens when the user launches the application. All the required components should start from the main activity.

AndroidManifest.xml

Location: app/src/main/AndroidManifest.xml

It stores all the important information of the application. It tells the application all the components in the application, points out the main activity to start the application and also tell

other information like permission required or app name and icon used.

App-level Gradle file

Location: app/build.gradle.kts

This file tells the application what configuration it should be built. It tells details like min sdk, target sdk, compile sdk, adding libraries like Jetpack Compose and also defining the application ID.

Version Catalog

Location: gradle/libs.versions.toml

Keeps all the versions for all the libraries and dependencies used.

Strings.xml

Location: app/src/main/res/values/strings.xml

This file is used to store text used in the android application. Instead of hardcoding directly in the code, all the text can be stored here and used in that part of the code.

Rotation Observation:

There are two states being used in the application, one for storing the data in the text field and the other is for storing the current dimension index being displayed on the screen. When rotating the screen, both these states get resetted to its default state. So I notice that the OutlinedTextField becomes empty and the initial dimension gets displayed on the screen.

An Android application needs to preserve the state as it is very much possible for a user to accidentally rotate the screen which would reset all the states and the user would go back to the initial data for the application.

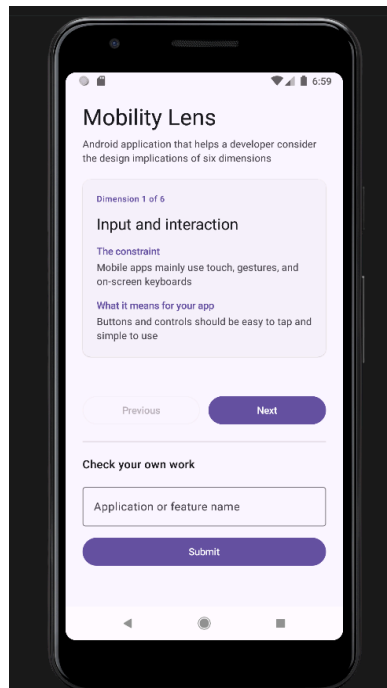
Mobile Design Decision:

One mobile design decision I implemented was the addition of a vertical scroll feature. When I first created all the components, I noticed some parts of the screen were not visible, especially if the keyboard is open on the screen or if the mobile phone is rotated. Without a vertical scroll some of the components won't be visible on the screen so it was important to add this feature. I implemented it on the MobileLensScreen.kt file where I included a vertical scroll modifier component to the main column.

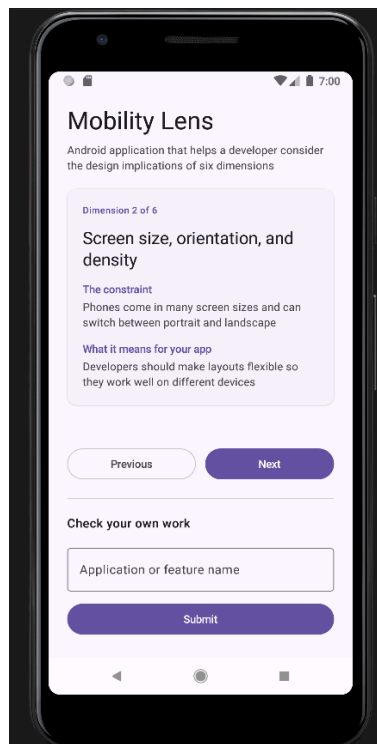
I don't think this scroll feature is needed for a desktop application as I feel a desktop screen size is larger compared to a mobile application and since the application had very less content I feel the entire application would have fitted properly on the desktop screen without a scroller.

Screenshots:

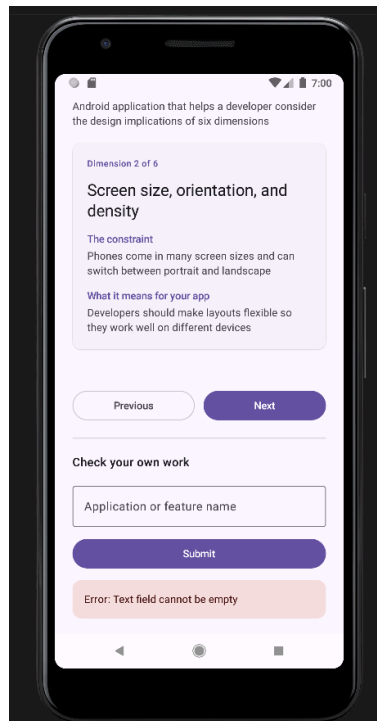
i) The running application on first dimension



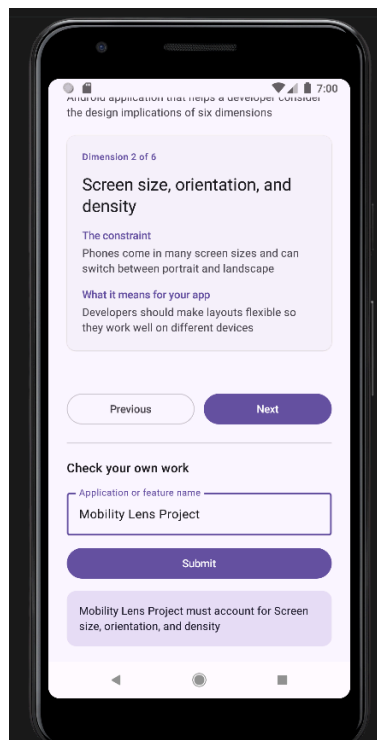
ii) A different dimension after user interaction



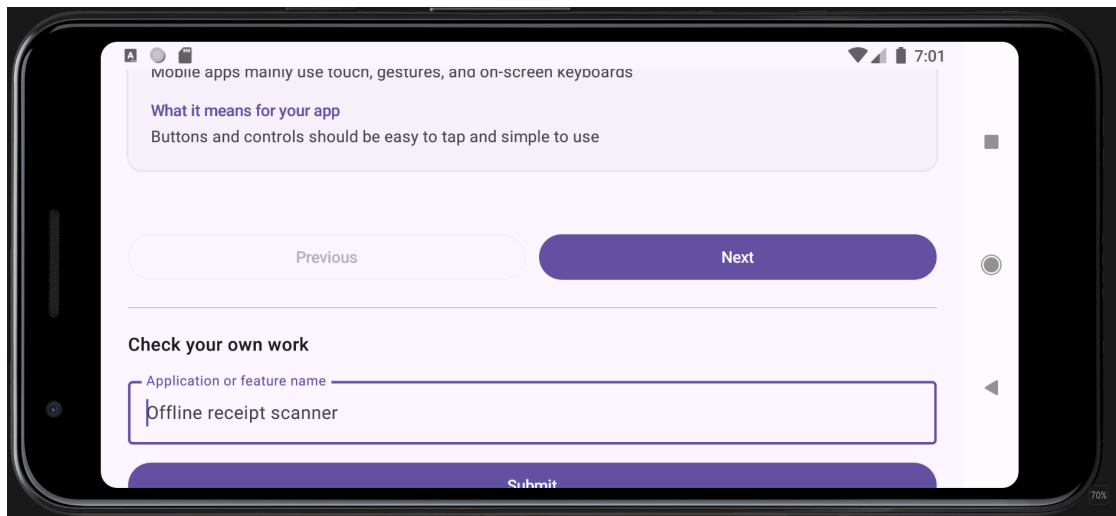
iii) The validation response to blank input



iv) A successful response to a valid input



v) Rotation Observation (Here you can see that the dimension data is the same as the first screenshot and the text field is blank since it has the placeholder text)



Collaboration and Generative AI Disclosure:

I worked on this assignment independently. However, I did use generative AI, specifically the Claude Opus 5 model, as a learning and support tool.

Since this was my first time working with Android Studio, I used the AI model extensively to help me understand the basic project structure after creating an Empty Activity project. I asked it to explain best practices for setting up different components in an Android application, how common Android templates are structured, and which UI components would be suitable for the design of my application.

I also used the model to help with debugging whenever I encountered errors. Although it generated some code suggestions, I did not simply copy everything it produced. I reviewed the suggestions and only used the parts that I felt would improve the application. I also made sure to understand all of the code suggested by the AI tool and how each part was being used within my application.

For example, the model suggested placing the Previous and Next navigation buttons below the OutlinedTextField, submit button, and information section because navigation controls are often placed near the bottom of a mobile interface. However, I decided to keep the Previous and Next buttons near the dimension details instead. I felt this made more sense because those buttons are specifically used to move between the different mobility dimensions and are not related to the OutlinedTextField section.

Prompts and suggestions

#	Prompt (summarised)	What the tool suggested
1	What minimum SDK is recommended for a typical Android project, and why?	API 26 (Android 8.0). Reasoning: ~95% device coverage, AndroidX requires 23+ so 23 is a hard floor, and 26 removes dual code paths for notification channels, adaptive icons, and background execution limits.
2	Best way to organise files in the Empty Activity template, and why?	Four source files: <code>MainActivity.kt</code> kept thin, <code>model/MobilityDimension.kt</code> , and <code>ui/</code> holding the screen and components. Advised the data class store <code>@StringRes</code> IDs rather than literal strings so all text stays in <code>strings.xml</code> . Advised against adding a <code>ViewModel</code> or navigation for a single screen.
3	(Attached starter project) What activities and components do I need? Step by step, I am new to Android Studio.	Corrected the plan of using multiple Activities: Compose uses one Activity and many composables, and the assignment requires a single screen. Gave a six-stage build order and a component checklist mapping each assignment requirement to a composable.
4	Should I create a new <code>ui</code> package or reuse <code>ui.theme</code> ?	<code>ui</code> already exists on disk and is hidden by the IDE's "Compact Middle Packages" setting. Advised creating files in <code>ui</code> and keeping <code>ui.theme</code> for theming only.
5	Show how the Stage 1 code is written, so I can see the syntax.	Provided a first-version <code>MobilityLensScreen</code> with a <code>Column</code> and two <code>Text</code> composables, the matching <code>MainActivity</code> , and an annotated explanation of <code>@Composable</code> , trailing lambdas, the <code>modifier</code> parameter convention, <code>stringResource</code> , and <code>dp</code> vs <code>sp</code> .
6	The navigator composable cannot change the index passed into it. How do I fix this?	Explained that Kotlin parameters are always <code>val</code> , and that mutating a copy would not trigger recomposition anyway. Suggested the state hoisting pattern: pass <code>onPreviousClick / onNextClick</code> lambdas and enabled flags instead of the index.
7	I cannot call <code>stringResource</code> inside the button's click handler.	<code>stringResource</code> is a composable and click handlers run after composition. Suggested reading a <code>Context</code> during composition and using <code>context.getString</code> in

		the lambda, with an alternative of capturing pre-read strings.
8	I cannot rename <code>com.example</code> to <code>com.jerinjoseph</code> in the Project panel.	Turn off "Compact Middle Packages" to expose example as its own node, then Refactor → Rename choosing "package". Also update namespace and <code>applicationId</code> in <code>build.gradle.kts</code> manually and rebuild.
9	(Pasted all four composables) Review my UI and make it more readable.	Flagged missing <code>verticalScroll</code> , <code>DimensionCard</code> not using an actual Card, feedback not clearing on navigation, a discarded <code>modifier</code> parameter, and stacked padding. Suggested typography hierarchy, labels above the description and implication, a "Dimension X of 6" indicator, a labelled text field, conditional feedback rendering, and error-coloured feedback. Also advised disabling <code>dynamicColor</code> .
10	I get a warning: "Querying resource values using <code>LocalContext.current</code> ".	Explained it is a lint warning about resources not re-reading on configuration change. Suggested replacing <code>LocalContext.current</code> with <code>LocalResources.current</code> .
11	Best way to set up Git through the Android Studio IDE?	Connect a GitHub account in Settings, then Git → GitHub → Share Project on GitHub, which initialises, commits, creates the remote, and pushes in one step.
12	I want to commit everything except the Gradle builds.	Distinguished build outputs (exclude) from build scripts (must commit). Confirmed the template <code>.gitignore</code> already excludes <code>/build</code> , <code>.gradle</code> , and <code>local.properties</code> , so no change was needed.
13	What do min, target and compile SDK mean, and why do they matter?	<code>compileSdk</code> acts at build time and sets which APIs can be written, <code>minSdk</code> acts at install time and sets the device floor, <code>targetSdk</code> acts at run time and selects which OS behaviours apply.